A Project Report for Design and Analysis of Algorithms

Birla Institute Of Technology and Science Pilani
Hyderabad Campus

Design and Analysis of Algorithms (CS F364)

Under the supervision of
Prof. Apurba Das & Prof. Tathagata Ray

Submitted By: **Group 28**

Sr No.	Name	ID Number
1	AKSHITH VUPPALA	2023A7PS0032H
2	ROHAN REDDY DEVARAPALLI	2023A7PS0138H
3	ADITHYA NAMA	2023A7PS0171H
4	VENU SAI YELESVARAPU	2023A7PS0149H
5	SATVIK ADERLA	2023A7PS0167H

Individual Contribution

AKSHITH VUPPALA (2023A7PS0032H)

- Implemented Algorithm 1 (Exact) from the *Efficient Algorithms for Densest Subgraph Discovery* paper.

ROHAN REDDY DEVARAPALLI (2023A7PS0138H)

- Implemented Algorithm 1 (GREEDY) from the *Flowless* paper.

ADITHYA NAMA (2023A7PS0171H)

- Managed input parsing and output formatting, and contributed to the preparation of the project report.

VENU SAI YELESVARAPU (2023A7PS0149H)

- Implemented Algorithm 4 (CoreExact) from the *Efficient Algorithms for Densest Subgraph Discovery* paper.

SATVIK ADERLA (2023A7PS0167H)

- Implemented Algorithm 2 (GREEDY++) from the *Flowless* paper.

Contents

1 Introduction **3**

2 Setup Instructions **4**
2.1 Execution Prerequisites 4
2.2 Execution Steps 4

3 Program Outputs **5**
3.1 Dataset 1: email-Enron 5
3.2 Dataset 2: wiki-Vote 6
3.3 Dataset 3: as-Skitter 7

4 Flow Algorithm Usage Choice **8**

1 Introduction

Finding the densest subgraph involves a strict trade-off between mathematical optimality and computational feasibility. We evaluated two distinct methodologies on three undirected networks: Wiki-Vote, Email-Enron, and AS-Skitter.

First, from *Efficient Algorithms for Densest Subgraph Discovery*, we implemented Algorithm 1 (Exact) and Algorithm 4 (CoreExact). For these exact algorithms, we specifically set $h = 3$ to compute the triangular (3-clique) density. This approach shrinks the graph via k-core decompositions before applying parametric maximum flow. Second, from *Flowless: Extracting Densest Subgraphs Without Flow Computations*, we implemented Algorithm 1 (GREEDY) and Algorithm 2 (GREEDY++). These algorithms bypass flow computations entirely, relying instead on iterative degree-based peeling.

Algorithmic Preference: Greedy vs. Exact

For large-scale graphs, greedy approximations are strongly preferred. While exact flow methods guarantee optimality, their worst-case complexity scales poorly, often stalling on dense graphs. Conversely, greedy approaches provide a 1/2-approximation in near-linear time. Our C++ implementations processed massive datasets like Wiki-Vote efficiently, proving that the memory and speed advantages of greedy peeling heavily outweigh the practical necessity of an exact mathematical optimum.

Algorithm	**Worst-Case Complexity**	**Mechanism**
Exact	$\mathcal{O}(V^2 E \log V)$	Max-flow (Dinic's) on full graph
CoreExact	$\mathcal{O}((V+E) + V_c^2 E_c \log V_c)$	k-core reduction + max-flow (Dinic's)
GREEDY	$\mathcal{O}(V+E)$	Single-pass degree peeling
GREEDY++	$\mathcal{O}(T \cdot (V+E))$	Multi-pass peeling (load balancing)

Table 1: Complexity Comparison of Implemented Algorithms

Variables: V, E (Original graph vertices/edges); V_c, E_c (Reduced k-core vertices/edges, where $V_c \ll V, E_c \ll E$); T (Number of GREEDY++ passes).

The sizes of Wiki-Vote, Email-Enron, and AS-Skitter is as follows

Dataset	**Nodes**	**Edges**
wiki-Vote	7,115	103,689
email-Enron	36,692	367,662
as-skitter	1,696,415	11,095,298

Table 2: Dataset Characteristics: Nodes and Edges Count

2 Setup Instructions

2.1 Execution Prerequisites

Before executing the script, verify that the following file structure is in the directory:

```
├── 1_flow1.cpp
├── 2_flow4.cpp
├── 3_greedy.cpp
├── 4_greedypp.cpp
├── Makefile
├── README.md
├── runall.sh
└── testcases/
    ├── as-skitter.txt
    ├── email-Enron.txt
    └── wiki-Vote.txt
```

Figure 1: Directory Structure

2.2 Execution Steps

To execute the analysis, follow these steps in the terminal:

1. Grant execution permission

   ```
   chmod +X runall.sh
   ```

2. Run the script for all outputs

   ```
   ./runall.sh
   ```

3. To get outputs only for one testcase, run the following

   ```
   ./runall.sh <path to testcase>
   ```

3 Program Outputs

3.1 Dataset 1: email-Enron

Exact (h=3)

```
Density                 : 404.456186
Number of Nodes         : 388
Number of Edges         : 14088
First 5 Sorted Nodes    : 56 , 72 , 74 , 75 , 76
Time Taken              : 25.6352s
Peak Memory Usage       : 177.05 MB
```

CoreExact (h=3)

```
Density                 : 404.456186
Number of Nodes         : 388
Number of Edges         : 14088
First 5 Sorted Nodes    : 56 , 72 , 74 , 75 , 76
Time Taken              : 2.5870s
Peak Memory Usage       : 85.04 MB
```

GREEDY

```
Algorithm: Greedy
Density                 : 37.344144
Number of Nodes         : 555
Number of Edges         : 20726
First 5 Sorted Nodes    : 27 , 46 , 53 , 54 , 56
Time Taken              : 0.1167s
Peak Memory Usage       : 15.36 MB
```

GREEDY++

```
Algorithm: Greedy++
Density                 : 37.344144
Number of Nodes         : 555
Number of Edges         : 20726
First 5 Sorted Nodes    : 27 , 46 , 53 , 54 , 56
Time Taken              : 0.1045s
Peak Memory Usage       : 15.37 MB
```

Algorithm	Time Taken	Peak Memory	Density	Nodes	Edges
Exact	25.6352s	177.05 MB	404.456186	388	14088
CoreExact	2.5870s	85.04 MB	404.456186	388	14088
GREEDY	0.1167s	15.36 MB	37.344144	555	20726
GREEDY++	0.1045s	15.37 MB	37.344144	555	20726

Table 3: Statistics for email-Enron Testcase

3.2 Dataset 2: wiki-Vote

Exact (h=3)

```
Density                 : 504.334764
Number of Nodes         : 466
Number of Edges         : 20520
First 5 Sorted Nodes    : 11 , 15 , 24 , 36 , 68
Time Taken              : 18.3862s
Peak Memory Usage       : 132.20 MB
```

CoreExact (h=3)

```
Density                 : 504.334764
Number of Nodes         : 466
Number of Edges         : 20520
First 5 Sorted Nodes    : 11 , 15 , 24 , 36 , 68
Time Taken              : 4.3816s
Peak Memory Usage       : 96.52 MB
```

GREEDY

```
Density                 : 46.275414
Number of Nodes         : 846
Number of Edges         : 39149
First 5 Sorted Nodes    : 6 , 11 , 15 , 20 , 24
Time Taken              : 0.0458s
Peak Memory Usage       : 13.40 MB
```

GREEDY++

```
Density                 : 46.279042
Number of Nodes         : 835
Number of Edges         : 38643
First 5 Sorted Nodes    : 6 , 11 , 15 , 20 , 24
Time Taken              : 0.0392s
Peak Memory Usage       : 13.23 MB
```

Algorithm	Time Taken	Peak Memory	Density	Nodes	Edges
Exact	18.3862s	132.20 MB	504.334764	466	20520
CoreExact	4.3816s	96.52 MB	504.334764	466	20520
GREEDY	0.0458s	13.40 MB	46.275414	846	39149
GREEDY++	0.0392s	13.23 MB	46.279042	835	38643

Table 4: Statistics for wiki-Vote Testcase

3.3 Dataset 3: as-Skitter

Exact (h=3)

```
Density                 : 3366.622328
Number of Nodes         : 421
Number of Edges         : 37391
First 5 Sorted Nodes    : 801 , 1027 , 1039 , 1080 , 1122
Time Taken              : 972.9784s
Peak Memory Usage       : 7546.20 MB
```

CoreExact (h=3)

```
Density                 : 3366.622328
Number of Nodes         : 421
Number of Edges         : 37391
First 5 Sorted Nodes    : 801 , 1027 , 1039 , 1080 , 1122
Time Taken              : 37.1809s
Peak Memory Usage       : 1715.42 MB
```

GREEDY

```
Density                 : 89.117096
Number of Nodes         : 427
Number of Edges         : 38053
First 5 Sorted Nodes    : 801 , 1039 , 1080 , 1122 , 1169
Time Taken              : 7.7980s
Peak Memory Usage       : 695.65 MB
```

GREEDY++

```
Density                 : 89.400862
Number of Nodes         : 464
Number of Edges         : 41482
First 5 Sorted Nodes    : 801 , 1027 , 1039 , 1080 , 1122
Time Taken              : 5.8626s
Peak Memory Usage       : 695.99 MB
```

Algorithm	Time Taken	Peak Memory	Density	Nodes	Edges
Exact	972.9784s	7546.20 MB	3366.622328	421	37391
CoreExact	37.1809s	1715.42 MB	3366.622328	421	37391
GREEDY	7.7980s	695.65 MB	89.117096	427	38053
GREEDY++	5.8626s	695.99 MB	89.400862	464	41482

Table 5: Statistics for as-skitter Testcase

4 Flow Algorithm Usage Choice

To compute the maximum flow for the **Exact** and **CoreExact** algorithms detailed in the work on Densest Subgraph Discovery, we implemented Dinic's algorithm rather than the Edmonds-Karp algorithm. This selection is justified by Dinic's strictly superior worst-case time complexity, which is critical for maintaining performance when processing massive, highly connected graphs.

Algorithm	**Time Complexity**	**Path Finding Mechanism**
Edmonds-Karp	$\mathcal{O}(VE^2)$	Single shortest augmenting path (BFS only)
Dinic's Algorithm	$\mathcal{O}(V^2E)$	Simultaneous blocking flows (BFS + DFS)

Table 6: Theoretical Time Complexity Comparison of Maximum Flow Algorithms

Theoretical Execution Time Estimation

To contextualize the performance improvement provided by Dinic's algorithm, we formulate an estimation model for the Edmonds-Karp execution time (T_{EK}) based on the recorded execution time of Dinic's algorithm (T_{Dinic}).

By analyzing the ratio of their worst-case theoretical bounds, we can isolate the scaling factor driven by network density:

$$\text{Ratio} = \frac{\mathcal{O}(VE^2)}{\mathcal{O}(V^2E)} = \frac{E}{V} \tag{1}$$

Assuming the constant factors hidden by the asymptotic notation are roughly equivalent for dense graphs, the estimated execution time for Edmonds-Karp scales proportionally to the average node degree (E/V):

$$T_{EK} \approx T_{Dinic} \times \left(\frac{E}{V}\right) \tag{2}$$

Applying this estimation model to as-skitter which consists of $V = 1,696,415$ vertices and $E = 11,095,298$ edges—and using our recorded Dinic's algorithm execution time of $T_{Dinic} = 941$ seconds, we can project the theoretical execution time for Edmonds-Karp:

$$T_{EK} \approx 941 \text{ s} \times \left(\frac{11,095,298}{1,696,415}\right) \tag{3}$$

$$T_{EK} \approx 941 \text{ s} \times 6.54 \approx 6154.5 \text{ s}$$

Converting this result yields approximately 1.71 hours. This estimation demonstrates that Edmonds-Karp would take nearly two hours to process the same graph that Dinic's algorithm solved in roughly 15 minutes. This drastic disparityclearly dictates the necessity of Dinic's algorithm for the dataset.